

Borla

Testing Report

Student: **George Boadu Junior**

Student ID: **22427354**

Course: CSCD 602 — Advanced Software Engineering, University of Ghana

Document: Testing_Report.pdf · Version 1.0 · 13 August 2026

1. Test strategy

Four layers, in the order they were actually run during the build:

1. **Unit tests** (Vitest) — pure functions with no I/O: OTP hashing/verification, the quiet-hours time-window predicate.
2. **Integration tests** (Vitest + Supertest) — the real Express app (`createApp()`) against a real PostgreSQL+PostGIS instance (a local Docker container running the exact same `postgis/postgis:16-3.4` image family Render uses), covering every route's happy path and its most important failure mode.
3. **Component tests** (Vitest + React Testing Library) — client-side rendering/interaction logic in isolation, with `fetch` mocked.
4. **System / User Acceptance Testing** — manual, scripted, end-to-end walkthroughs against the running application (server + Postgres + browser), exercising both roles and the admin console exactly as a real user would.

Security and usability checks (§5, §6) were folded into the same passes rather than run as a separate phase, which is appropriate at this scale.

2. Automated test results

2.1 Server (`npm run test -w server`)

```
✓ test/integration/broadcasts.test.ts (4 tests)
✓ test/integration/reviews.test.ts (7 tests)
✓ test/integration/requests.test.ts (8 tests)
✓ test/integration/admin.test.ts (5 tests)
✓ test/integration/auth.test.ts (13 tests)
✓ test/unit/redisRateLimit.test.ts (3 tests)
✓ test/unit/quietHours.test.ts (4 tests)
✓ test/unit/phone.test.ts (4 tests)
✓ test/unit/otp.test.ts (3 tests)
✓ test/unit/sms.test.ts (3 tests)

Test Files 10 passed (10)
Tests      54 passed (54)
Duration   ~50s
```

`redisRateLimit.test.ts` and `phone.test.ts` are new since the Redis/BullMQ migration (Technical_Debt_Plan.md TD-05) and the phone-normalisation fix (D-07); `broadcasts.test.ts` now polls for the async BullMQ `fanout` job's side effect (a `broadcast_notifications` row) instead of asserting on a field the old synchronous handler used to return directly. `requests.test.ts` grew again for the cancel/arrival lifecycle (FR-28/FR-29); `reviews.test.ts` for review-in-request-context (FR-31) and the rating-aggregate staleness fix (D-11).

2.2 Client (`npm run test -w client`)

```
✓ src/pages/StatusChip.test.tsx (3 tests)

Test Files 1 passed (1)
Tests      3 passed (3)
```

Total: 57/57 automated tests passing at time of submission (client dropped from 5 to 3 — `ReviewForm.tsx` and its test were retired along with the double-blind review model, TD-15). Re-run with `npm test` from the repository root (requires a reachable Postgres+PostGIS and Redis — see `README.md`).

3. Test case log

Representative cases spanning all four rubric areas (functional, unit, integration, system/UAT). "Actual result" reflects the real run, including seven real defects directly caught by an inline test-case row (T-08, T-19, T-39, T-48, T-55, T-57, T-60 — the full set of thirteen defects, several found other ways, is in §4).

#	Test case	Layer	Expected result	Actual result
T-01	Request an OTP for a brand-new phone number with no <code>role</code>	Integration	<code>400</code> — role required to sign up	<code>400</code> returned with actionable message
T-02	Request an OTP, verify with the correct code	Integration	New user created, <code>access</code> / <code>refresh</code> issued	User created with <code>role='household'</code> , tokens returned
T-03	Verify with an incorrect 6-digit code	Integration	<code>400</code> , attempt counter incremented	<code>400</code> returned
T-04	Re-use an already-consumed OTP code	Integration	<code>400</code> — rejected as already used	<code>400</code> returned
T-05	<code>GET /auth/me</code> with no bearer token	Integration	<code>401</code>	<code>401</code> returned
T-06	<code>GET /auth/me</code> with a valid token	Integration	<code>200</code> with role-specific profile	<code>200</code> , <code>profile.alert_radius_m</code> present for household
T-07	Admin login with a non-admin phone number	Integration	<code>401</code>	<code>401</code> returned
T-08	(defect) OTP-verify signup path for a brand-new number	Integration (found while writing T-02)	New user row created with the requested role	First implementation threw <code>role_missing</code> inside a dead tree branch and never created the user — see §4
T-09	Collector attempts to go online before admin	Integration	<code>403</code>	<code>403</code> returned

#	Test case	Layer	Expected result	Actual result
	verification			
T-10	Verified + online collector, household broadcasts nearby	Integration	Fan-out notifies ≥ 1 collector; pin appears in that collector's <code>/pins/nearby</code>	<code>notifiedCollectors: 1</code> ; pin present with correct <code>distance_m</code>
T-11	Household creates a second broadcast while one is active	Integration	409	409 returned
T-12	Collector role attempts to create a broadcast	Integration	403 (role guard)	403 returned
T-13	Household requests an offline collector	Integration	409	409 returned
T-14	Full request lifecycle: requested $\rightarrow$ seen $\rightarrow$ accepted, contact reveal timing	Integration	Phone numbers absent pre-accept, present post-accept	Confirmed via direct field assertion pre/post
T-15	Reject a request after it was already accepted	Integration	409 , status stays <code>accepted</code> (not resurrected to <code>rejected</code>)	409 returned, status unchanged
T-16	A collector who wasn't the request's target tries to accept it	Integration	409 (conditional update matches 0 rows)	409 returned
T-17	Review submitted against a request that is not yet accepted	Integration	400	400 returned
T-18	Duplicate review on the same accepted request	Integration	409 (unique constraint)	409 returned
T-19	(defect) Reply to a review before it is moderation-visible	Integration (found while writing this test)	400 — cannot reply to a non-visible review	First implementation only checked <code>subject_id</code> , not <code>status</code> reply on a still-pending review — see §4
T-20	<i>(superseded by T-58 — TD-15 removed the one-reply-per-review cap this test was checking)</i> Second reply attempt on an already-replied review	Integration	409 (one reply per review)	409 returned, at the time
T-21	Admin verifies a collector; collector can now go online; action appears in audit log	Integration	200 $\rightarrow$ 200 ; <code>audit_log</code> contains <code>verify_collector</code>	All three confirmed
T-22	Admin suspends a household; suspended user's <code>/auth/me</code> is blocked; reinstate restores access	Integration	403 while suspended, 200 after reinstatement	Confirmed
T-23	Admin edits an unknown config key	Integration	404	404 returned
T-24	Full system walkthrough : household broadcast $\rightarrow$ collector sees pin on map $\rightarrow$ household clears it	System/UAT (manual, live server)	Pin visible then gone from collector's feed	Verified against the running app with real curl/browser sessions
T-25	Full system walkthrough : direct request $\rightarrow$ accept $\rightarrow$ contact reveal $\rightarrow$ review both sides $\rightarrow$ double-blind release fires within the 2-minute sweep window	System/UAT (manual, live server)	Both reviews become visible simultaneously , rating aggregates recompute correctly	<code>rating_avg</code> / <code>rating_count</code> confirmed via direct DB query after tl
T-26	Manual moderation fallback (no <code>GEMINI_API_KEY</code> set)	System/UAT	Submitted reviews appear in <code>/admin/moderation/queue</code> $\rightarrow$ awaitingManual , not silently published	Confirmed — both test reviews appeared, required explicit admin ap
T-27	Quiet-hours predicate across a midnight-wrapping window	Unit	Inside/outside window classified correctly at boundaries	4/4 boundary cases correct
T-28	OTP hash/verify round-trip and mismatch rejection	Unit	Correct code verifies true, wrong code false	Both confirmed
T-29	<code>StatusChip</code> renders the correct label for every request status + an unknown fallback	Component	Exact label text present for each of 5 statuses + fallback	Confirmed
T-30	<code>ReviewForm</code> treats a 409 (already reviewed) the same as success	Component	Renders the thank-you state, not an error banner	Confirmed
T-31	Production client build (<code>vite build</code>) and server build (<code>tsc</code>) both compile clean	Build/System	Zero TypeScript errors, bundle produced	Both confirmed (<code>tsc --noEmit</code> clean; <code>dist/</code> produced)
T-32	GiantSMS phone-number normalisation (<code>+233...</code> / <code>233...</code> $\rightarrow$ local <code>0...</code> form)	Unit	Both prefixes normalise to the same local number; an already-local number is untouched	3/3 cases correct

#	Test case	Layer	Expected result	Actual result
T-33	GiantSMS end-to-end request against the real funded account, production	System (live)	Gateway accepts the send request	<code>POST /api/auth/otp/request</code> against <code>https://borla.onrender.com</code> <code>{"delivered":true}</code> — GiantSMS accepted the request at the HTTP 200; handset receipt not visually confirmed (test number is a placeholder, not a real phone)
T-34	Unified sign-in: a new phone number gets an OTP with no role needed upfront; the code screen then requires role+name	Integration + scripted screenshot	<code>isNewUser:true</code> ; role/name fields appear only after the code step, submit creates the account	Confirmed both via <code>auth.test.ts</code> and a real headless-browser wall
T-35	Unified sign-in: an existing (non-admin) number logs straight in from the code screen, no role/name prompt	Integration + scripted screenshot	<code>isNewUser:false</code> ; verify returns tokens with no extra fields required	Confirmed
T-36	Unified sign-in: an admin's phone number is auto-detected and routed to a password prompt instead of an OTP	Integration + scripted screenshot	<code>requiresPassword:true</code> , no OTP issued; UI shows the password form directly	Confirmed both ways
T-37	PWA service worker registers and activates on the production build	System (headless browser against dist/)	<code>navigator.serviceWorker.getRegistrations()</code> returns an active registration scoped to <code>/</code>	Confirmed: <code>{"scope":"http://localhost:5175/","active":true,"scriptURL":"http://localhost:5175/build/service-worker.js","link_rel":"manifest"}</code> present in the DOM
T-38	Seeded demo phone numbers always get the fixed <code>DEMO_OTP</code> , are exempt from the rate limit, and never trigger a real SMS attempt (D-06)	Integration	8 rapid <code>/otp/request</code> calls all succeed and return <code>devOtp:"482913"</code> , <code>delivered:false</code>	Confirmed via <code>auth.test.ts</code> ; also re-confirmed directly against prod
T-39	(defect) An already-registered phone number typed without the leading <code>+</code> (e.g. <code>233200000001</code>)	Integration + manual (found by the user against the live app, D-07)	Recognised as the same existing account, logs straight in	First implementation stored/looked up the raw string, so <code>+2332000000001</code> / <code>2332000000001</code> / <code>02000000001</code> were three different things — see §4
T-40	<code>normalizePhone</code> collapses all three Ghanaian phone-number input forms (<code>+233...</code> , <code>233...</code> , <code>0...</code>) to one canonical string	Unit	All three forms produce an identical result	4/4 cases correct (<code>phone.test.ts</code>)
T-41	Redis-backed presence: a collector going online is visible to a nearby household via <code>GEOSEARCH</code> , not a Postgres scan	Integration + manual (real local Redis)	<code>ZCARD geo:collectors</code> and <code>EXISTS presence:{id}</code> both reflect the online collector immediately	Confirmed via direct Redis inspection alongside the <code>/collectors/online</code> endpoint
T-42	Redis-backed per-collector notification rate limit caps fan-out messages independently per collector	Unit	Requests beyond the cap are rejected for that collector only; a different collector is unaffected	3/3 cases correct (<code>redisRateLimit.test.ts</code>)
T-43	Redis-backed OTP request rate limit caps requests per phone number	Unit	The (cap+1)th request for the same phone is rejected	Confirmed (<code>redisRateLimit.test.ts</code>)
T-44	Broadcast fan-out runs as an async BullMQ job, not inline in the request handler	Integration	<code>POST /broadcasts</code> returns before fan-out completes; a <code>broadcast_notifications</code> row appears shortly after, once the <code>fanout</code> job runs	Confirmed via <code>waitFor()</code> polling in <code>broadcasts.test.ts</code> — row appears in the poll window, HTTP response contains no notification count
T-45	Live Gemini moderation classification against the real provisioned key (D-08 fix)	Manual (real key, <code>server/src/ai/moderation.ts</code>)	A genuine, non-fallback <code>allow / flag / block</code> verdict, not a <code>null</code> that degrades to the manual queue	<code>classifyText("Great pickup, right on time, very professional")</code> returned <code>{"verdict":"allow","categories":[""],"piiFound":false,"confidence":0.99,"reason":"The text is a legitimate review with no presence of abuse, harassment, or hate speech."}</code> — a real classification
T-46	Gemini model-ID resilience: the candidate-list fallback skips a 404'd model instead of failing closed entirely	Manual (real key, direct HTTP probe of 6 candidate IDs)	At least one candidate in the list resolves	<code>gemini-2.5-flash</code> / <code>gemini-2.0-flash</code> → 404; <code>gemini-3.6-flash</code> / <code>gemini-3.5-flash</code> / <code>gemini-3.7-flash</code> / <code>gemini-flash-latest</code> → 200 OK; <code>gemini-3.6-flash</code> first and succeeds immediately
T-47	FR-27: <code>GET /requests/mine</code> exposes the right location to the right side, gated the same way as contact reveal	Integration	Collector always sees the household's pickup point (already shared at request creation); household sees no collector location before acceptance, then the real one after	Confirmed in <code>requests.test.ts</code> — <code>collector_lon/lat</code> are <code>null</code> until <code>accept</code> ; <code>household_lon/lat</code> present before <code>accept</code>
T-48	(defect) <code>GET /reviews/mine</code> — an author can see their own review regardless of visibility, distinct from <code>GET /users/{id}/reviews</code> (visible-only)	Integration + manual (D-10, reported by the user)	Author sees the review immediately with an honest status (<code>pending / visible</code>); a stranger's "mine" list never contains someone else's review	Confirmed live: a fresh review submitted, then approved and released to production (~15s in production, well under the 2-min interval) — the mechanism works; the gap was that neither side had noticed that it was working

#	Test case	Layer	Expected result	Actual result
T-49	Double-blind release fires quickly once both sides have genuinely reviewed each other on the same request	System (live, production)	Both reviews become visible to each other shortly after the second side submits	Two fresh reviews submitted on the same accepted request in production passed AI moderation within ~10s and were mutually visible within 1 sweep cycle (~15s observed)
T-50	FR-28: a household can cancel its own pending request (previously impossible — only accept/reject existed, both collector-only actions)	Integration	200 , status becomes cancelled with cancelled_by: 'household' ; a second cancel attempt is 409	Confirmed in requests.test.ts
T-51	FR-28: a collector can cancel an accepted request; a stranger cannot cancel a request they're not part of	Integration	Collector cancel 200 ; stranger cancel 409 (conditional update matches zero rows, same idempotency pattern as accept/reject)	Confirmed
T-52	FR-29: a collector's live position reaching the pickup radius marks the request arrived; a position well outside it does not	Integration	Heartbeat ~5-7km away leaves arrived_at null; a heartbeat at the exact pickup point sets it, status stays 'accepted'	Confirmed via direct ST_DWithin check against requests.location
T-53	FR-29: once arrived, cancel is no longer offered — there's nothing left to back out of	Integration	Cancel attempt after arrived_at is set returns 409	Confirmed
T-54	FR-31: GET /requests/:id/reviews shows both directions of review in the context of the request they belong to	Integration	Author sees mine regardless of status; the subject sees theirs only once visible; a non-party gets 403	Confirmed in reviews.test.ts
T-55	(defect) Removing a previously-visible review recomputes the subject's rating_avg / rating_count instead of leaving them stale	Integration (found while implementing FR-31, D-11)	Admin POST /reviews/:id/remove on a visible 5-star review drops the subject's rating_count back to 0, not left at the pre-removal value	First implementation only updated the aggregate when the release sweep itself made a review visible — an admin removal afterwards never re-ran the computation — see §4
T-56	StatusChip renders 'On the way' / 'Arrived' distinctly for an accepted request, and a label for cancelled	Component	Each status/arrived combination shows its own label	3/3 cases correct (StatusChip.test.tsx)
T-57	(defect) TD-15: a review now joins the shared thread the moment it clears moderation, instead of waiting for the other side to also review	Integration	A review approved via moderation appears in GET /users/:id/reviews immediately — no dependency on the other party's own review	Confirmed in reviews.test.ts — visible right after status='visible' second review on the request at all
T-58	TD-15: either party can reply to a review, more than once each	Integration	Both the review's author and its subject can each post multiple replies; a non-party gets 403	3 successive replies from both sides confirmed in reviews.test.ts correctly rejected
T-59	TD-15: GET /requests/:id/reviews returns one identical shared thread to both parties, in order	Integration	Both callers see the same messages array (review + reply, chronological); each caller's own not-yet-visible submission only shows to them via mine	Confirmed — both sides' responses matched exactly once cleared; prior the author saw their own mine.status='pending'
T-60	(defect) D-13: an admin can manually approve a reply stuck awaiting moderation, not just a review	Integration	POST /admin/replies/:id/approve makes it visible ; a second attempt on the same reply is 404 (already resolved)	Confirmed in admin.test.ts
T-61	D-13: a Gemini call failure with a key configured is distinguished from "no key at all"	Manual (real key, direct classifyText call in isolation)	The exact text of a reply stuck in production's manual queue classifies successfully when called directly, supporting "transient failure," not "genuinely unmoderatable content"	classifyText("Thank you so much, see you again!", "review") returned a real { "verdict": "allow", "confidence": 0.99, ... } on job's context

4. Defects found during development

Defect	Where	Root cause	Corrective action
D-01	server/src/utils/jwt.ts	jsonwebtoken 's TypeScript types don't accept a plain string for expiresIn (expects its own StringValue literal union)	Cast through jwt.SignOptions["expiresIn"] explicitly rather than loosening the type globally
D-02	server/src/modules/auth/routes.ts (OTP verify)	An early draft wrapped user-creation in a transaction block that threw a sentinel error (role_missing) to short-circuit — but the catch swallowed it without ever inserting the user, so first-time signup silently returned an "unexpected signup state" error	Rewritten as a straight-line if (!user) { ...INSERT... } with no transaction/sentinel-error indirection; covered by T-02/T-08

Defect	Where	Root cause	Corrective action
D-03	<code>server/src/seed.ts</code>	First draft referenced a non-existent <code>verified_note</code> column on <code>collectors</code> (copy-paste artefact) wrapped in a silent <code>.catch()</code> fallback that masked the real error	Removed the phantom column and the <code>.catch()</code> swallow entirely; seed now fails loudly if it ever breaks again
D-04	<code>server/src/modules/reviews/routes.ts</code> (reply endpoint)	Only checked <code>review.subject_id === user.id</code> , not <code>review.status === 'visible'</code> , so a reply could be posted on a still-hidden/pending review	Added the status check; covered by T-19
D-05 (security)	<code>server/src/modules/auth/routes.ts</code> (<code>/auth/otp/request</code> , <code>/auth/otp/verify</code>)	Neither endpoint checked <code>users.role</code> — an admin account (meant to require phone+password) could also be logged into via the plain OTP flow, defeating the two-tier auth model entirely. Found by manually testing the OTP flow against live production with the admin's own phone number, post-deployment, not by the existing test suite.	Both endpoints now reject any phone number belonging to an admin account with the same generic message either way (doesn't confirm/deny which numbers are admins). Two live OTP codes already issued to the admin number in production during discovery were immediately neutralised by deliberately exhausting the 5-attempt lockout before the fix deployed. Added two regression tests (<code>refuses to send an OTP to an admin's phone number</code> , <code>refuses to verify an OTP into an admin account even if a code somehow exists</code>) so this can't silently regress.
D-06	<code>server/src/modules/auth/routes.ts</code> (<code>/otp/request</code>)	Once real GiantSMS delivery (D-05's neighbour, TD-02) started reporting <code>delivered:true</code> , the two seeded demo phone numbers — which are arbitrary, not real handsets — would have had their OTP silently "delivered" into a gateway with no phone behind it, hiding the fallback <code>devOtp</code> and locking the examiner out of the graded demo accounts entirely. Found by re-reading the deployment credentials file after wiring up real SMS, before it ever actually caused a lockout.	<code>DEMO_PHONES</code> / <code>DEMO_OTP</code> added to <code>server/src/seed.ts</code> : the two seeded numbers always get the same fixed, documented code, are exempt from the OTP rate limit, and never trigger a real SMS attempt regardless of gateway state. One regression test added (<code>the seeded demo household number always gets the fixed DEMO_OTP... , 8 rapid requests all succeed and return the fixed code</code>).
D-07	<code>server/src/modules/auth/routes.ts</code> (<code>phoneSchema</code> , both OTP endpoints)	Phone numbers were stored and looked up as the raw string the client sent, with no normalisation — <code>+233200000001</code> , <code>233200000001</code> , and <code>0200000001</code> were three different rows to Postgres, so an already-registered user who typed their number without the leading <code>+</code> looked brand new and was sent through the sign-up (role + name) path instead of logging straight in. Found by the user testing the seeded household account (233200000001, no +) against the live app.	Added <code>server/src/utils/phone.ts</code> (<code>normalizePhone</code>), wired into <code>phoneSchema</code> via Zod's <code>.transform()</code> so every route that accepts a phone number normalises it before it ever reaches a query or an insert. Four new unit tests (<code>phone.test.ts</code>) plus regression coverage in <code>auth.test.ts</code> confirming the same number in all three input forms resolves to one account.
D-08	<code>server/src/ai/moderation.ts</code> (<code>MODEL</code> constant)	The moderation pipeline was hardcoded to <code>gemini-2.5-flash</code> . Once the user provisioned a real <code>GEMINI_API_KEY</code> , live calls started returning <code>404 — this model is no longer available to new users</code> for that account's tier, which the fail-closed design (correctly) turned into every review silently landing in the manual queue instead of surfacing a visible error. Found via production logs after the key was set.	First fix (switching to a single hardcoded <code>gemini-flash-latest</code>) still couldn't be confirmed live. Root-caused properly by probing six candidate model IDs directly against the real key: <code>gemini-2.5-flash</code> / <code>gemini-2.0-flash</code> both 404, <code>gemini-3.6-flash</code> / <code>gemini-3.5-flash</code> / <code>gemini-3.7-flash</code> / <code>gemini-flash-latest</code> all work. Rewrote <code>classifyText</code> to try an ordered candidate list and cache whichever one succeeds per process, instead of betting on one guessed ID — confirmed working end-to-end locally (T-45/T-46).
D-09	<code>client/src/components/Avatar.tsx</code> , <code>client/src/styles/tokens.css</code>	Two related defects, both found by the \$6 screenshot pass rather than by reading the code: (1) the <code>Avatar</code> component referenced <code>.avatar / .avatar.g</code> classes that had never actually been added to <code>tokens.css</code> , so initials rendered as bare unstyled text with no circular background; (2) the initials helper took the first character of the <i>last</i> whitespace-separated token without checking it started with a letter, so <code>"Ama (Osu)"</code> produced <code>"A("</code> .	Added the missing <code>.avatar / .avatar.g</code> rule block; filtered the initials helper to letter-led words only. Re-screenshotted to confirm both fixes.
D-10	<code>client/src/pages/Profile.tsx</code> , <code>client/src/components/ReviewForm.tsx</code>	The double-blind release mechanism itself was working correctly (confirmed by T-49), but nothing in the UI <i>said</i> so: <code>GET /users/:id/reviews</code> only ever returns <code>status='visible'</code> rows by design, and there was no way for the author of a review to see their own submission anywhere — not on Profile, not after the initial "submitted" moment. A one-sided review (the common case until the other party also reviews) looked indistinguishable from a silently-broken or lost one. Reported by the user ("why is approved reviews and given reviews not seen by either party") after testing the flow themselves.	Added <code>GET /reviews/mine</code> (any status, author-only) and a "Reviews I've given" section on Profile showing each review's real status (<code>Awaiting moderation</code> / <code>Approved</code> — waiting on the other side... / <code>Public</code> / etc.); rewrote <code>ReviewForm</code> 's post-submit message to explain the hold instead of a bare "thanks".
D-11	<code>server/src/jobs/workers.ts</code> (<code>recomputeRatingAggregate</code>), <code>server/src/modules/admin/routes.ts</code>	<code>rating_avg</code> / <code>rating_count</code> were only ever recomputed from inside the review-release sweep, for the reviews <i>it</i> just released. Admin actions that change a review's status outside that sweep — removing a visible review directly, or resolving a moderation flag as "remove"/"clear" — updated <code>reviews.status</code> but never touched the subject's aggregate, leaving it stale (e.g. a removed 5-star review would keep inflating <code>rating_count</code> forever). Found while adding FR-31's review-in-context view , reasoning through every path that changes review visibility rather than just the sweep.	Exported <code>recomputeRatingAggregate</code> from <code>jobs/index.ts</code> ; both <code>/admin/reviews/:id/remove</code> and <code>/admin/moderation/flags/:id/resolve</code> now call it for the affected subject after updating status. Also hardened the function itself to reset to zero (not leave the previous value) when a subject ends up with no visible reviews at all. Regression test added (T-55).
D-12	<code>server/src/jobs/workers.ts</code> (<code>reviewReleaseSweep</code> , now removed), <code>server/src/modules/reviews/routes.ts</code>	The double-blind release model meant the two parties to the <i>same</i> request could see different content on the <i>same</i> request card at the same time — whichever side hadn't reviewed yet saw nothing, making the card look inconsistent or broken rather than "waiting on the other person". Reported directly by the user ("why can't	This is TD-15, not a small patch: removed the reciprocal-release sweep entirely; a review or reply now goes <code>visible</code> the moment it individually clears moderation (same as replies always worked); <code>review_replies</code> one-per-review cap and subject-only restriction were both dropped so either party can reply, any number of times; <code>GET /requests/:id/reviews</code> now returns one identical <code>messages</code>

Defect	Where	Root cause	Corrective action
		both users see the same comments on the request card") together with a concrete ask: an open reply chain/chat, either party, AI-moderated.	thread to both callers. Documented as a deliberate trade-off (dropped collusion-resistance in exchange for consistency and a real chat) rather than a silent regression — see Technical_Debt_Plan.md TD-15. Tests: T-57–T-59.
D-13	<code>server/src/jobs/workers.ts</code> (<code>processModerate</code>), <code>server/src/modules/admin/routes.ts</code>	Found live in production while verifying D-12/TD-15: a transient Gemini failure (not "no key configured", an actual call failure — plausibly rate-limiting from this session's own heavy testing) left two genuine replies stuck in the manual queue indefinitely. Root cause: <code>processModerate</code> returned quietly on a null verdict instead of throwing, so the <code>moderate</code> queue's own <code>attempts:3 /exponential-backoff</code> config (<code>jobs/queues.ts</code>) never engaged — a transient failure got exactly one attempt, forever. Compounding it: there was no admin "Approve" action for a stuck <i>reply</i> at all (only reviews had one), so once stuck, a reply had zero recovery path.	<code>processModerate</code> now distinguishes "no key configured" (permanent, returns cleanly, no point retrying) from "the call itself failed" (throws, letting BullMQ's existing retry/backoff actually run) — confirmed locally that the exact stuck reply text classifies fine in isolation, supporting transient failure as the cause. Added <code>POST /admin/replies/:id/approve</code> , mirroring the review one, plus its "Approve" button in the admin UI for reply-type queue items (previously review-only). Regression test added (server/test/integration/admin.test.ts).

All thirteen were caught by testing immediately after (or, for D-05/D-06/D-07/D-08/D-10/D-12, well after) the corresponding feature — eight by the automated suite (D-11 by reasoning through the code while building an unrelated feature, D-13 by directly observing stuck production queue items while manually verifying D-12), five by deliberately exercising the live deployed app or reading its logs (D-05 by me re-testing production myself; D-07, D-10, and D-12 reported back by the user; D-08 surfaced in production logs once the Gemini key went live), and D-09 by a scripted screenshot pass rather than by reading the code — direct evidence for why TD-10 (test depth) is listed as "scheduled," not "critical": the practice works, it just hasn't been extended to every corner of the app, including production behaviour, yet. D-10 and D-11 make a related point: a *correctly working* backend mechanism (the double-blind release sweep; the rating aggregate) can still fail users if either its state is never surfaced (D-10) or it's only kept correct along one of several paths that touch it (D-11) — functional correctness on the happy path and correctness everywhere the same data can change are different bars. D-12 is a step further still: the mechanism was working *exactly as designed*, and the design itself was the problem — no amount of additional testing against the original spec would have caught it, because the spec (double-blind release) was what the user was objecting to. D-13, found while manually re-verifying D-12's own fix in production, is a reminder that verification itself can surface new defects — the retry/backoff infrastructure existed since TD-05 but had never actually been exercised until real usage hit it. Some defects are only found by putting the actual feature in front of the actual person it's for, and some are only found by watching your own fix operate for real.

5. Security testing

Check	Method	Result
Every mutating/reading-sensitive route requires a valid JWT	Automated (T-05, T-12, T-16, T-22) + manual <code>curl</code> without a token	Consistently <code>401 / 403</code>
SQL injection surface	Code review — every query in <code>server/src/modules/**</code> uses parameterised <code>\$1..\$n</code> placeholders, none concatenate user input into SQL	No string-built SQL found
OTP brute-force / spam	Code review + manual test + unit (T-43)	5-per-10-minutes request cap (now Redis-backed, <code>checkOtpRateLimit</code> , per-phone rather than per-IP — Technical_Debt_Plan.md TD-05) and a 5-attempt verify cap are enforced (<code>otp/routes.ts</code>)
Role escalation (collector calling household-only routes, etc.)	Automated (T-12) + manual <code>curl</code> across all role-gated routes	Consistently <code>403</code>
Contact information leakage pre-accept	Automated (T-14)	Phone fields verified absent from the JSON payload itself (not just hidden in the UI) before acceptance
Secrets handling	Code review	JWT secrets and the Gemini key are read from environment variables only, never committed (<code>.env</code> is git-ignored, <code>.env.example</code> has placeholders)
Admin account reachable via the weaker OTP path (bypassing phone+password)	Manual testing against the live production deployment	Confirmed exploitable (D-05) — fixed immediately, redeployed, and re-verified <code>400</code> on both endpoints for the admin's number

6. Usability testing

A design-fidelity pass against the approved `borla_UI_design.html` mockups: the brand mark (pin + marigold radar dot), the line-icon system, coloured initial avatars, and the signature animated "broadcast ring" were all re-implemented in the real app (not just approximated with emoji) and verified with real, scripted screenshots (Puppeteer driving headless Chrome against the running dev server, logged in as the seeded demo accounts) rather than eyeballing the code.

D-09 (found this way) — see §4 — covers both the missing avatar styling and the broken initials logic that the screenshot pass surfaced.

A second usability pass, prompted directly by user feedback rather than a screenshot diff: admin-facing copy had leaked internal spec references (`design §17` , Technical Debt Plan TD-01/TD-02) into user-visible text, and a live-ops legend spelled out colour names in prose ("gold = active pins, green = online collectors") instead of showing an actual colour swatch. Both are informational-only, so no defect ID or regression test was warranted — fixed directly in `AdminDashboard.tsx / Login.tsx / auth/routes.ts` and the new `.legend-dot` style (`tokens.css`), re-verified by re-reading the rendered JSX rather than a screenshot.

Beyond that: tap targets $\geq 44\text{px}$ (`.btn` padding), icon-first primary actions, a single primary action per screen, and colour contrast matching the approved palette (green/marigold/coral on a warm paper background) — confirmed manually in-browser at mobile viewport widths (390px, 414px) and desktop.

7. Performance testing

Out of scope at pilot scale by design (see SRS NFR discussion). Two checks performed: (1) the PostGIS `ST_DWithin` queries that Postgres still runs as the durable mirror use the `GIST` index created in `001_init.sql` (`EXPLAIN on broadcasts_loc_gix` confirmed an index scan, not a sequential scan, even against the small seeded dataset); (2) since

Technical_Debt_Plan.md TD-05, the actual hot-path nearby-collector/nearby-pin lookups run against Redis `GEOSEARCH` (`server/src/redis/presence.ts`) instead of Postgres on every request — confirmed functionally correct (T-41) but not load-tested at real traffic volume.

8. What was NOT tested (honestly stated, ties to Technical_Debt_Plan.md TD-10)

Visual confirmation of a GiantSMS text actually arriving on a real handset (T-33 confirmed the gateway *accepts* the request in production; the seeded demo number is a placeholder, not a live phone someone was watching); a live Gemini moderation call *specifically against the deployed production process* (T-45/T-46 confirm the fixed code works end-to-end with the real key run locally — production just needs to be confirmed running the same code with the same env var, not re-derived from scratch); concurrent double-accept under real network race conditions (only sequential-call idempotency is proven); load/stress testing; cross-browser automated testing (manual only); the four `CollectorHome` / `HouseholdHome` / `AdminDashboard` / `Profile` React pages have no component tests yet, only the two smaller components (`StatusChip` , `ReviewForm`).